

Diseño Microelectrónico Avanzado — MUISE

Práctica 1

Verificación de una FIFO síncrona parametrizable

Banco de pruebas pre-UVM en SystemVerilog

Joan San Bartolomé Ferri
Miquel Àngel Llaneras Gelabert

Profesor: Rafael Gadea

6 de octubre de 2026

Índice

1. Resumen y veredicto	2
2. Contenido de la entrega y ejecución	2
2.1. Estructura del directorio	2
2.2. Scripts para la demostración	3
2.3. Cómo leer el resultado	4
3. Diseño bajo verificación	4
3.1. FIFO de referencia del profesor	4
3.2. FIFO sintetizable propia	4
4. Arquitectura del banco de pruebas	5
4.1. Interfaz, modports y clocking blocks	6
4.2. Generación aleatoria con restricciones (RCSG)	6
4.3. Driver, entorno y test	7
4.4. Monitor	7
4.5. Scoreboard: predictor y evaluador	7
4.6. Cobertura funcional	8
4.7. Inyección controlada de errores	8
5. Resultados	8
5.1. Simulación nominal de la FIFO propia	8
5.2. Cobertura funcional	10
5.3. Comparación con el test dirigido del profesor	11
5.4. Detección de errores inyectados	12
5.5. FIFO del profesor	14
5.6. Parametrización	14
6. Conclusiones	16

1. Resumen y veredicto

En esta práctica hacemos de equipo de verificación que tiene que dar (o no) el visto bueno a un diseño de otro equipo. El diseño bajo verificación (DUV) es una FIFO síncrona con reset asíncrono, borrado síncrono, contador de palabras y *flags* de vacía y llena. Con el mismo banco de pruebas hemos verificado dos implementaciones: `FIFO_no_sintetizable`, el modelo de referencia del profesor (una cola de SystemVerilog), y `FIFO_sintetizable`, la nuestra, diseñada a partir del diagrama ASM del enunciado sobre la memoria de doble puerto `ram_dp`.

El banco es pre-UVM y orientado a objetos: interfaz con *modports* y *clocking blocks*, generación aleatoria con restricciones (RCSG), monitor, *scoreboard* con modelo de referencia, cobertura funcional y un test que se guía por la propia cobertura. Está todo parametrizado con `WIDTH` (ancho de dato) y `DEPTH` (profundidad).

Tabla 1: Resumen de resultados.

Comprobación	Resultado
Scoreboard, FIFO propia (8×32)	2 550 comparaciones de las cuatro salidas y 0 errores (RESULTADO: EXITO)
Scoreboard, FIFO del profesor (8×32)	2 550 comparaciones y 0 errores, con los mismos estímulos que la FIFO propia
Cobertura funcional	100 % (141 de 141 bins) en 2 550 ciclos; el test fijo de antes se quedaba en 99,79 % con 10 000
Aserciones de protocolo del DUV	100 % cubiertas, sin fallos
Inyección de errores	Los tres modos probados (FLAG , ESTADO y DATOIDLE) se detectan en el flanco siguiente a la inyección
Parametrización	8×32, 16×16 y 4×64 con las dos FIFOs (Regresion.do): los seis casos sin errores y con 100 % de cobertura
Veredicto	Damos <code>FIFO_sintetizable</code> por APTA

Cambios respecto a lo sugerido por el profesor. Como piden las instrucciones, los indicamos aquí y los detallamos en la sección 4.

- El predictor mantiene la cola ideal sugerida, pero guarda cada resultado esperado en un *mailbox* propio, compara todos los ciclos (también los que no tienen operación) y recuerda el último dato leído (sección 4.5).
- El covergroup del guion lo hemos pasado a una clase que se muestrea desde el monitor, con los bins de nivel definidos con `DEPTH` y con dos combinaciones imposibles ignoradas en `estados_normales` (sección 4.6).
- El test, después de las fases de llenado y vaciado, sigue con ráfagas aleatorias hasta llegar al 100 % de cobertura (sección 4.3).

2. Contenido de la entrega y ejecución

2.1. Estructura del directorio

Entregamos un único `.zip` con la estructura del proyecto. Todos los scripts se lanzan desde la carpeta `sim/`.

```
Practicas-DMA/
|-- src/ Fuentes SystemVerilog
```

```
| |-- FIFO_sintetizable.sv DUV propio (ASM + ram_dp)
| |-- ram_dp.sv Memoria de doble puerto
| |-- FIFO_no_sintetizable.sv FIFO de referencia del profesor
| |-- FIFO_tb_v0.sv Test dirigido del profesor
| |-- fifo_if.sv Interfaz, modports y clocking blocks
| |-- FIFO_top_duv.sv Envoltorio del DUV con puerto de interfaz
| |-- FIFO_tb.sv Top de verificacion
| |-- utilidades_pkg.sv Package con todas las clases
| |-- RCSG_base/subir/bajar.sv Generadores aleatorios con restricciones
| |-- FIFO_driver.sv, FIFO_Monitor.sv, FIFO_Scoreboard.sv,
| |-- FIFO_Coverage.sv, FIFO_Transaction.sv,
| |-- FIFO_Enviroment.sv, FIFO_Testaleatorio1.sv
| '-- FIFO_inyeccion_errores.sv Inyector de errores (bind)
|-- sim/
| |-- filelist.txt Orden de compilacion
| |-- Comp.do Compilacion
| |-- Sim.do Simulacion (FIFO, parametros y top configurables)
| |-- SimErr.do Simulacion con inyeccion de errores
| |-- Regresion.do Las dos FIFOs con tres combinaciones de parametros
| '-- wave.do Configuracion de ondas
'-- .vscode/tasks.json Tareas de VS Code (Compile, Simulate...)
```

2.2. Scripts para la demostración

Todo se lanza desde `sim/` con QuestaSim 2025.1_2 (win64). Primero se compila con `Comp.do` y después cada situación que pide el enunciado (FIFO propia, FIFO del profesor, cambio de parámetros, test dirigido y detección de errores) se demuestra con una orden de la tabla 2. No hace falta editar ningún fuente: la FIFO y los parámetros se eligen con variables que `Sim.do` traduce a opciones `-g` de `vsim`.

Tabla 2: Órdenes para la demostración (se lanzan desde `sim/`).

Orden	Qué demuestra
<code>vsim -batch -do "do Comp.do; quit -f"</code>	Compila todos los ficheros de <code>filelist.txt</code> .
<code>vsim -do Sim.do</code>	Test aleatorio sobre nuestra FIFO, 8×32 .
<code>vsim -do "set PROFESOR 1; do Sim.do"</code>	El mismo test sobre la FIFO del profesor.
<code>vsim -do "set WIDTH 16; set DEPTH 16; do Sim.do"</code>	Otros parámetros (se pueden combinar con PROFESOR).
<code>vsim -do "set top FIFO_tb_v0; do Sim.do"</code>	Test dirigido del profesor con el covergroup del guion.
<code>vsim -do "set ERR FLAG; do SimErr.do"</code>	Inyección de un error (DATO, FLAG, USEDW, ESTADO o DATOIDLE).
<code>vsim -c -do Regresion.do</code>	Las dos FIFOs con 8×32 , 16×16 y 4×64 , con un resumen final.

`Sim.do` simula hasta el final del test (`run -all`) y al terminar genera el informe HTML de cobertura en `sim/covhtmlreport/`. `Regresion.do` compila, lanza las seis simulaciones en modo consola, guarda cada una en `reg_<fifo>_<W>x<D>.log` y termina con una línea **RESULTADO** por caso. Las variables (PROFESOR, WIDTH, DEPTH, top) se quedan definidas en la sesión de QuestaSim, así que para volver a la configuración por defecto conviene abrir una sesión nueva.

Desde VS Code. Las mismas órdenes están como tareas en `.vscode/tasks.json`: *Compile* (Ctrl+Shift+B), *Simulate*, *Simulate FIFO profesor*, *Test dirigido profesor*, *Simulate con errores* y

Regresión. Simulate con errores abre un desplegable para elegir el modo de error, y todas compilan antes de simular.

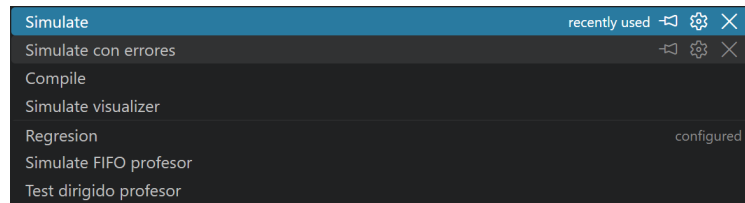


Figura 1: Menú *Run Task* de VS Code con las tareas del proyecto.

2.3. Cómo leer el resultado

Cada simulación acaba con el informe del *scoreboard* (figura 7), cuya última línea es **RESULTADO:** EXITO o **RESULTADO:** FALLO. El fallo lo emitimos con `$error` para que QuestaSim lo cuente en su resumen de errores. También damos FALLO si no se ha comparado ninguna transacción, para que un banco mal conectado no pase por bueno.

3. Diseño bajo verificación

3.1. FIFO de referencia del profesor

`FIFO_no_sintetizable` implementa la FIFO con una cola acotada de SystemVerilog (`[$:DEPTH]`): escribir es `push_front`, leer es `pop_back`, y `USE_DW` y los *flags* salen del tamaño de la cola. Tiene dos aserciones concurrentes, `sobrellenado` y `sobrevaciado`, que avisan si se escribe con la FIFO llena o se lee con ella vacía. No se puede sintetizar, pero define el comportamiento esperado.

3.2. FIFO sintetizable propia

Nuestra `FIFO_sintetizable` tiene exactamente las mismas entradas y salidas que el modelo anterior y sigue el diagrama ASM del enunciado (figura 2). La unidad de control es una máquina con los estados `VACIO`, `OTROS` y `LLENO`; los *flags* son salidas de Moore (solo dependen del estado) y las órdenes a la ruta de datos (`wren`, `rden`, incrementar o decrementar `USE_DW`) dependen además de `READ` y `WRITE`. En la ruta de datos están los punteros `countw` y `countr`, que vuelven a 0 al llegar a `DEPTH-1`, el contador `USE_DW` y la memoria `ram_dp`, con escritura y lectura síncronas. Si con la FIFO vacía llegan a la vez lectura y escritura, el dato de entrada pasa directamente a la salida (`DATA_OUT <= DATA_IN`) sin ocupar la memoria, como indica el ASM, y un multiplexor elige entre esta ruta directa y la salida de la RAM.

Se pasa a `LLENO` al escribir con `USE_DW == DEPTH-1` y se vuelve a `VACIO` al leer con `USE_DW == 1`, así que todo depende de `DEPTH`. `RESET_N` (asíncrono) y `CLEAR_N` (síncrono) llevan la FIFO a `VACIO` y ponen `DATA_OUT` a 0, como el modelo del profesor. También hemos conservado las aserciones de protocolo del modelo de referencia.

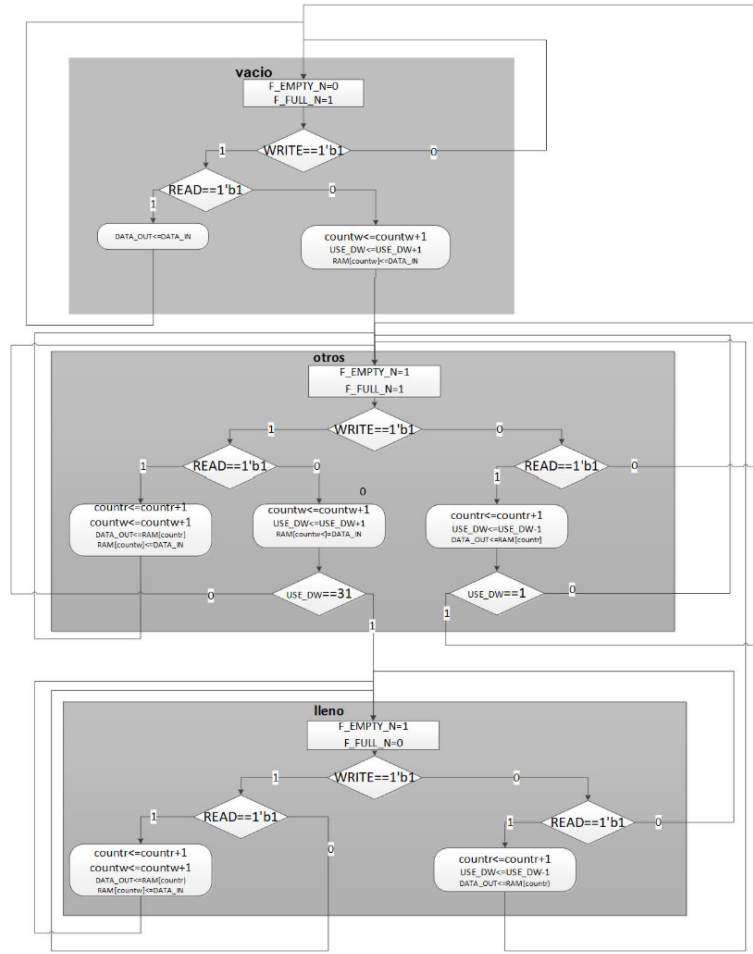


Figura 2: Diagrama ASM de la FIFO del enunciado, implementado en FIFO_sintetizable.

La `ram_dp` que se nos dio escribía `mem[waddress] <= data_in`, pero el puerto se llama `data`, así que la corregimos para que compilara. Además, `ram_dp` calcula el ancho de dirección como $\$clog2(mem_depth-1)$ y la FIFO como $\$clog2(DEPTH)$. Coinciden para cualquier potencia de 2 (16, 32, 64...), que son las profundidades que usamos en la regresión.

4. Arquitectura del banco de pruebas

En el banco separamos la *controlabilidad* (generar y aplicar estímulos) de la *observabilidad* (capturar y comprobar respuestas). Cada componente va en su propio fichero y las clases se agrupan en el package `utilidades_pkg`, que las declara con `typedef class` y las incluye en orden para que la compilación no dependa de referencias cruzadas. En la figura 3 se ve cómo se conectan.

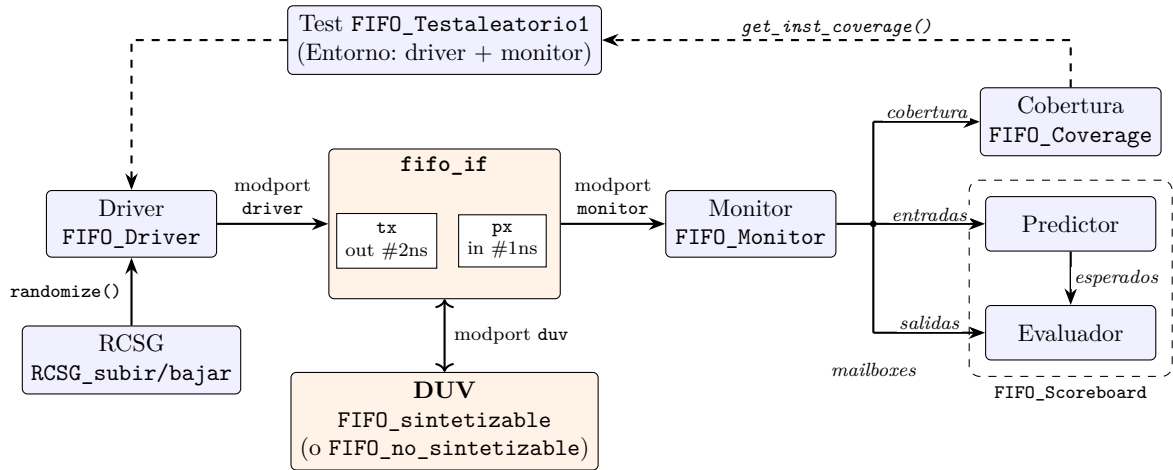


Figura 3: Arquitectura del banco de pruebas. Las flechas discontinuas son el lazo de control por cobertura (el test consulta la cobertura y decide qué secuencia aplica el driver).

4.1. Interfaz, modports y clocking blocks

`fifo_if` agrupa todas las señales de la FIFO y es el único punto de contacto entre las clases y el hardware. Tiene tres *modports*: `duv` (lo usa `FIFO_top_duv`, cuyo único puerto es la interfaz), `driver` y `monitor`. Con los *clocking blocks* fijamos cuándo se aplica y cuándo se muestrea cada señal, y así evitamos carreras entre el banco y el DUV:

Tabla 3: Clocking blocks de `fifo_if` (reloj de 100 ns).

Bloque	Evento	Skew	Uso
tx	posedge clk	output #2 ns	El driver aplica <code>rd_en</code> , <code>wr_en</code> , <code>data_in</code> , <code>rst_s</code> 2 ns después del flanco.
px	posedge clk	input #1 ns	El monitor muestrea todas las señales 1 ns antes del flanco, ya estables.
neg_event	negedge clk	output #2 ns	El driver aplica el reset asíncrono <code>rst_a</code> lejos del flanco activo.

`FIFO_tb` genera el reloj, instancia la interfaz, el DUV (a través de `FIFO_top_duv`) y el objeto test, y le pasa la interfaz virtual. Sus parámetros `DATA_WIDTH`, `DEPTH` y `FIFO_PROFESOR` se propagan a todo el banco. Con el último, `FIFO_top_duv` elige en un `generate` qué FIFO instancia, la nuestra o la del profesor. Así, para cambiar de DUV o de tamaño basta con pasar `-g` a `vsim`, sin editar ni recompilar el código.

4.2. Generación aleatoria con restricciones (RCSG)

`RCSG_base` tiene los campos aleatorios `data_in`, `rd_en` y `wr_en` y un modelo `static` del nivel de llenado (`use_dw`) que se actualiza en `post_randomize()`. Con ese modelo bastan dos restricciones para que el estímulo sea siempre legal:

```
constraint rd_c { ((use_dw == 0) && (wr_en == 0)) -> (rd_en == 0); } // no leer vacía
constraint wr_c { ((use_dw == DEPTH) && (rd_en == 0)) -> (wr_en == 0); } // no escribir
                             llena
```

Las clases hijas solo cambian, con `dist`, la distribución de operaciones: `RCSG_subir` favorece las escrituras (nada 10 %, solo escritura 40 %, solo lectura 20 %, ambas 30 %) y `RCSG_bajar` las lecturas (10/20/40/30 %). Como `use_dw` es estático, las dos ven el mismo nivel aunque las alternemos.

4.3. Driver, entorno y test

El driver aplica el reset inicial (`rst_a` activo durante tres ciclos de reloj) y tiene dos secuencias, `secuencia_llenado` y `secuencia_vaciado`, que aleatorizan un objeto RCSG y lo aplican por el *clocking block* `tx`. El entorno solo construye driver y monitor con sus interfaces virtuales.

El test, `FIFO_Testaleatorio1`, arranca en paralelo monitor, cobertura, predictor y evaluador, y aplica:

1. 1 000 ciclos de llenado y 1 000 de vaciado.
2. Un bucle dirigido por cobertura en ráfagas de 50 ciclos. En cada una elige un nivel objetivo aleatorio entre 0 y `DEPTH` y usa la secuencia de llenado si la FIFO está por debajo o la de vaciado si está por encima. Tras cada ráfaga consulta `get_inst_coverage()`, y termina al llegar al 100 % o a 10 000 ciclos.
3. Imprime la cobertura de cada *coverpoint* y *cross*, el valor de `$get_coverage()` y el informe del scoreboard.

El bucle lo añadimos porque con un test de duración fija la FIFO pasaba el 83 % del tiempo a dos palabras o menos de vacía o llena y apenas visitaba los niveles intermedios; con niveles objetivo se recorren todos y se cierra la cobertura (sección 5.2).

4.4. Monitor

En cada flanco del *clocking block* `px` el monitor captura todas las señales en una `FIFO_Transaction` y envía tres copias independientes (`clone()`) para que ningún hilo modifique la transacción de otro. La cobertura recibe la transacción tal cual, el predictor las entradas del ciclo y el evaluador las salidas de la FIFO en el flanco siguiente, que son la respuesta a esas entradas. Lo hacemos en todos los ciclos, también en los que no hay lectura ni escritura, para comprobar que en ellos la FIFO no cambia.

4.5. Scoreboard: predictor y evaluador

El predictor reproduce la FIFO ideal con una cola, igual que el modelo del profesor, y calcula el valor esperado de las cuatro salidas (`DATA_OUT`, `F_EMPTY_N`, `F_FULL_N` y `USE_DW`). El evaluador empareja cada salida real con su esperada y las compara con aserciones inmediatas.

Cambios respecto al predictor sugerido. En la versión inicial el evaluador leía el esperado de una variable compartida que el predictor reescribía cada ciclo, y nos dimos cuenta de que, como los dos procesos se despiertan con el mismo flanco, podían acceder a ella en cualquier orden (condición de carrera). Ahora el predictor deposita cada esperado, clonado, en un *mailbox* y el evaluador los saca en orden FIFO, así que cada salida se compara siempre con su esperado, haga lo que haga el planificador. Además, el predictor recuerda el último dato leído (`ultimo_dato_out`) y `DATA_OUT` se compara en todos los ciclos, no solo en las lecturas. Por último, cada aserción suma aciertos o errores por salida y `report()` imprime los contadores y da un veredicto (sección 2).

```
case ({read_enable, write_enable})
  2'b01: if (FIFO_ideal.size() < DEPTH) FIFO_ideal.push_front(data_in);
  2'b10: ultimo_dato_out = FIFO_ideal.pop_back();
  2'b11: begin FIFO_ideal.push_front(data_in); // con FIFO vacía: paso directo
          ultimo_dato_out = FIFO_ideal.pop_back(); end
endcase
esperado.data_out = ultimo_dato_out;
esperado.use_dw = FIFO_ideal.size();
mailbox_predecidas.put(esperado.clone());
```

4.6. Cobertura funcional

FIFO_Coverage recibe las transacciones del monitor y las muestrea en un *covergroup* sin evento de disparo (`with function sample`). La tabla 4 resume su contenido.

Tabla 4: Modelo de cobertura funcional.

Elemento	Bins
<code>grado_llenado</code>	Un bin por nivel intermedio (1 a <code>DEPTH-1</code>), más vacía (0) y llena (<code>DEPTH</code>).
<code>flags</code>	Vacía, llena e intermedia; <code>illegal_bins</code> para vacía y llena a la vez.
<code>rw</code>	Solo lectura, solo escritura, ambas; el resto (<code>default</code>) no cuenta.
<code>estados_normales</code>	Cruce nivel $\times$ operación, ignorando escribir con la FIFO llena y leer con la vacía.
<code>casos_especiales</code>	Cruce operación $\times$ <i>flags</i> , con las mismas combinaciones ignoradas.

Cambios respecto al covergroup sugerido. Partimos del covergroup del guion, que se declara en un módulo y se muestrea con `@(posedge clock)` sobre las señales del DUV. Los coverpoints y crosses son los mismos, pero hemos cambiado tres cosas:

- Lo hemos pasado a una clase que recibe las transacciones del monitor por un *mailbox* y llama a `sample()`. De esta forma la cobertura se mide sobre lo mismo que comprueba el scoreboard y puede consultarse desde el test con `get_inst_coverage()`.
- Los bins de `grado_llenado` se definen con `DEPTH` (`[1:DEPTH-1]` y `DEPTH`) en lugar de 31 y 32, para que el modelo valga con cualquier profundidad.
- En `estados_normales` ignoramos, como ya hacía el guion en `casos_especiales`, escribir solo con la FIFO llena y leer solo con ella vacía. El generador nunca produce esas operaciones porque las restricciones las prohíben, y sin ignorarlas el cruce no podría llegar al 100 %.

El covergroup original, con su evento de muestreo, lo seguimos usando en `FIFO_tb_v0` para medir el test dirigido del profesor (sección 5.3), así que en el proyecto hay covergroups de los dos tipos.

4.7. Inyección controlada de errores

Para comprobar que el monitor y el scoreboard detectan fallos reales usamos el módulo `FIFO_inyeccion_errores`, que se engancha dentro de `FIFO_sintetizable` con `bind` sin modificar la FIFO ni el banco. Espera a un instante conocido (por ejemplo, lectura y escritura simultáneas con la FIFO a medias), fuerza una señal interna con `force` durante un ciclo y la suelta con `release`. El error se elige al lanzar la simulación con `+ERR=<modo>`; sin ese argumento el módulo no hace nada. Hay cinco modos: `DATO` (un bit de `DATA_OUT` en una lectura), `FLAG` (`F_FULL_N` activo sin estar llena), `USEDW` (contador desplazado), `ESTADO` (FSM forzada a `LLEN0`) y `DATOIDLE` (`DATA_OUT` cambia sin operación).

El `bind` lo pusimos en un módulo contenedor, `FIFO_inyeccion_bind`, que `SimErr.do` carga como segundo *top*, así que la simulación nominal no incluye el inyector.

5. Resultados

Salvo que digamos lo contrario, los resultados son con `WIDTH = 8` y `DEPTH = 32`.

5.1. Simulación nominal de la FIFO propia

En las primeras formas de onda del test aleatorio (figura 4) la FIFO se llena y se vacía alternando lecturas y escrituras, y nunca se escribe estando llena ni se lee estando vacía. La figura 5 ya es la FIFO sintetizable con el test final: en la fase de llenado llega a llena y se queda cerca de 32

palabras, y en la de vaciado baja a 0 y se mantiene cerca de vacía. El caso límite de lectura y escritura simultáneas con la FIFO vacía está en la figura 6, y el informe final del scoreboard, con 2 550 comparaciones de cada salida y ningún error, en la figura 7.

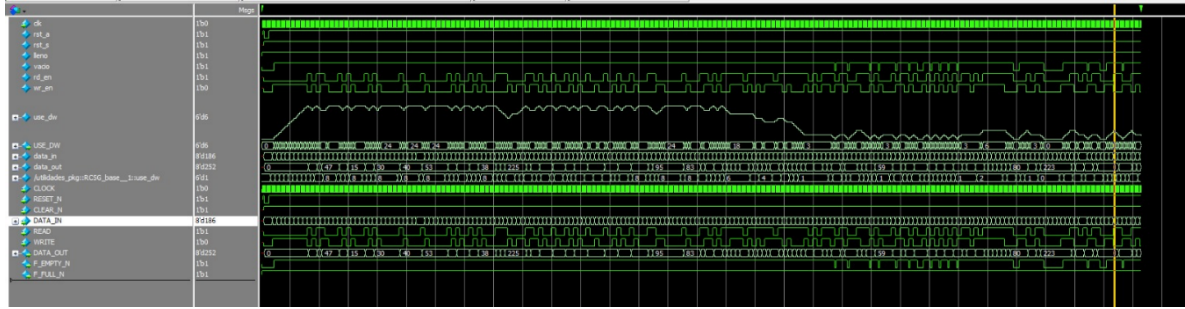


Figura 4: Formas de onda del primer test aleatorio con RCSG: nivel de llenado (`use_dw`), lecturas, escrituras y *flags*.

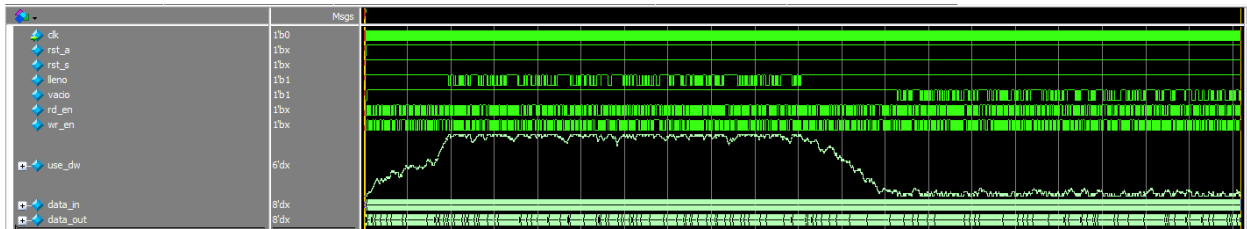


Figura 5: FIFO sintetizable con el test final. lleno y vacio son `F_FULL_N` y `F_EMPTY_N` en la interfaz, y `use_dw` está en formato analógico.

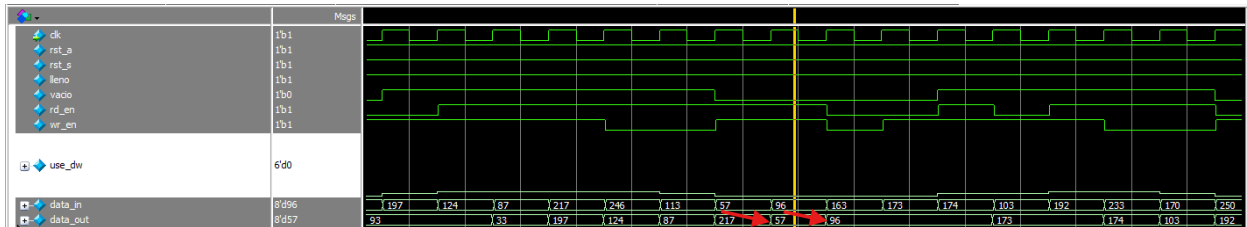


Figura 6: Lectura y escritura simultáneas con la FIFO vacía (`vacio` = 0, `rd_en` = `wr_en` = 1). Los datos 57 y 96 pasan de `data_in` a `data_out` en el flanco siguiente, mientras `use_dw` se queda a 0.

```
# ===== INFORME DEL SCOREBOARD =====
# Transacciones comparadas: 2550
# DATA_OUT   aciertos = 2550   errores =    0
# F_EMPTY_N   aciertos = 2550   errores =    0
# F_FULL_N    aciertos = 2550   errores =    0
# USE_DW      aciertos = 2550   errores =    0
# Esperados pendientes de comparar: 1
# RESULTADO: EXITO - 2550 comparaciones sin errores
# =====
# Simulacion finalizada exitosamente.
# ** Note: $finish      : ../src/FIFO_tb.sv(47)
# Time: 255750 ns  Iteration: 4  Instance: /FIFO_tb
```

Figura 7: Informe final del scoreboard en la simulación nominal, con 2 550 comparaciones por salida y **RESULTADO: EXITO**.

5.2. Cobertura funcional

Con el test de duración fija (5 000 ciclos de llenado y 5 000 de vaciado) la cobertura se quedaba en el 99,79 % (140 de 141 bins). Faltaba el bin `intermedio[22] × lectura_escritura`, porque los niveles intermedios recibían entre 6 y 30 muestras frente a unas 1 900 en cada extremo. El test dirigido por cobertura cierra ese hueco (tabla 5).

Tabla 5: Cobertura funcional por elemento.

Elemento	Test fijo (10 000 ciclos)	Test por cobertura
<code>grado_llenado</code>	100 %	100 %
<code>flags</code>	100 %	100 %
<code>rw</code>	100 %	100 %
<code>estados_normales</code>	98,96 %	100 %
<code>casos_especiales</code>	100 %	100 %
Total	99,79 %	100 % en 2 550 ciclos

En la tabla 5 se comparan los dos tests con el mismo covergroup. Con el test dirigido por cobertura todos los elementos llegan al 100 %, incluido el cruce `estados_normales`, que era el único incompleto.

En la figura 8 se ve cómo se llega. Tras las dos fases fijas (2 000 ciclos) y la primera ráfaga, la cobertura está en el 99,59 %; igual que en el test fijo, esas fases llevan la FIFO a los extremos y dejan sin visitar combinaciones de niveles intermedios. A partir de ahí cada ráfaga de 50 ciclos lleva la FIFO a un nivel objetivo aleatorio y recorre los niveles intermedios. La cobertura sube al 99,79 % en la séptima ráfaga y al 100 % en la undécima, así que el bucle solo añade 550 ciclos.

En total, el test necesita **2 550 ciclos para el 100 %**, cuatro veces menos que los 10 000 del test fijo, que encima se quedaba en el 99,79 %. La diferencia es que aquí la cobertura, además de medirse, guía los estímulos: el test mira lo que devuelve `get_inst_coverage()` para decidir qué secuencia de la RCSG aplicar y cuándo parar.

```
# [205550000] Cobertura = 99.59 % tras 50 ciclos (objetivo nivel 17)
# [210550000] Cobertura = 99.59 % tras 100 ciclos (objetivo nivel 17)
# [215550000] Cobertura = 99.59 % tras 150 ciclos (objetivo nivel 8)
# [220550000] Cobertura = 99.59 % tras 200 ciclos (objetivo nivel 19)
# [225550000] Cobertura = 99.59 % tras 250 ciclos (objetivo nivel 15)
# [230550000] Cobertura = 99.59 % tras 300 ciclos (objetivo nivel 30)
# [235550000] Cobertura = 99.79 % tras 350 ciclos (objetivo nivel 25)
# [240550000] Cobertura = 99.79 % tras 400 ciclos (objetivo nivel 28)
# [245550000] Cobertura = 99.79 % tras 450 ciclos (objetivo nivel 2)
# [250550000] Cobertura = 99.79 % tras 500 ciclos (objetivo nivel 14)
# [255550000] Cobertura = 100.00 % tras 550 ciclos (objetivo nivel 22)
# FIN DEL TEST
# grado_llenado      : 100.00 %
# estados_normales   : 100.00 %
# casos_especiales    : 100.00 %
# Cobertura total ($get_coverage): 100.00 %
```

Figura 8: Evolución de la cobertura en el transcript, con una línea por ráfaga de 50 ciclos y el nivel objetivo elegido, hasta el 100 %.

Path	Coverage	Count	Goal	Status
utilidades_pkg/FIFO_Coverage__1	100.00%	100	100.00...	✓
TYPE Cobertura_funcional	100.00%	100	100.00...	✓
CVP Cobertura_funcional::grado_llenado	100.00%	100	100.00...	✓
CVP Cobertura_funcional::flags	100.00%	100	100.00...	✓
CVP Cobertura_funcional::rw	100.00%	100	100.00...	✓
CROSS Cobertura_funcional::estados_normales	100.00%	100	100.00...	✓
CROSS Cobertura_funcional::casos_especiales	100.00%	100	100.00...	✓
INST \utilidades_pkg::FIFO_Coverage__1::Cobertura_funcional...	100.00%	100	100.00...	✓

Figura 9: Ventana *Covergroups* de QuestaSim con todos los coverpoints y crosses al 100 %.

Total Coverage (100%)

Overall Design Unit Coverage Summary:

Assertions	Covergroups
100%	100%

Coverage Summary by Design Unit:

Design Unit	Assertions	Covergroups	Total
work.FIFO_sintetizable	100%	-	100%
work.utilidades_pkg	100%	100%	100%

Figura 10: Informe HTML de cobertura (`coverage report -html`): 100 % en aserciones y covergroups.

Covergroups type work.utilidades_pkg::FIFO_Coverage__1/Cobertura_funcional

Summary

	Bins	Hits
Coverpoints	39	39
Crosses	102	102

coverpoints | **crosses**

Crosses	Bins	Hits	Misses	Goal	Coverage
estados_normales	97	97	0	100	100%
casos_especiales	5	5	0	100	100%

Figura 11: Informe HTML del covergroup: 39 de 39 bins en los coverpoints y 102 de 102 en los crosses (`estados_normales`, 97 de 97).

5.3. Comparación con el test dirigido del profesor

Para comparar con el test dirigido del profesor (`FIFO_tb_v0`, casos `test_A` a `test_D` sobre `FIFO_no_sintetizable`) añadimos a ese banco el covergroup del guion, que se muestrea en cada flanco de reloj. Tiene los mismos coverpoints y crosses que el nuestro; la única diferencia es que en `estados_normales` no ignora las dos combinaciones que el generador nunca produce, así que ese cruce tiene 99 bins en lugar de 97 (sección 4.6). Se lanza con `vsim -do "set top FIFO_tb_v0; do Sim.do"`.

Tabla 6: Cobertura del test dirigido del profesor frente a nuestro test aleatorio.

Elemento	Dirigido (FIFO_tb_v0)	Aleatorio por cobertura
grado_llenado	100 %	100 %
flags	100 %	100 %
rw	100 %	100 %
estados_normales	80,80 %	100 %
casos_especiales	100 %	100 %
Total	96,16 %	100 %

El test dirigido pasa por todos los niveles de llenado y por todos los *flags*, pero solo cubre 80 de los 99 bins de *estados_normales*: sus lecturas y escrituras simultáneas se concentran en unos pocos niveles (FIFO vacía, llena o tras un llenado parcial), así que no combina cada nivel intermedio con cada tipo de operación. Aunque descontáramos los dos bins inalcanzables, se quedaría en 80 de 97 (82,5 %). El test aleatorio cubre los 97 bins de ese cruce, de modo que supera al dirigido justo en lo que este no prueba: el comportamiento de la FIFO en cualquier nivel con cualquier operación.

Name	Coverage	Goal	% of Goal	Status	Included	Merge_instances	Get_inst_coverage
/FIFO_tb_v0	96.16%	100	96.16%	✓			
TYPE Cobertura_funcional	96.16%	100	96.16%	✓			auto(0)
CVP Cobertura_funcional::grado_llenado	100.00%	100	100.00%	✓			
CVP Cobertura_funcional::flags	100.00%	100	100.00%	✓			
CVP Cobertura_funcional::rw	100.00%	100	100.00%	✓			
CROSS Cobertura_funcional::estados_normales	80.80%	100	80.80%	✓			
CROSS Cobertura_funcional::casos_especiales	100.00%	100	100.00%	✓			
INST /FIFO_tb_v0/cbf	96.16%	100	96.16%	✓			0

Figura 12: Ventana *Covergroups* tras el test dirigido del profesor: 96,16 % en total y 80,80 % en *estados_normales*.

5.4. Detección de errores inyectados

Hemos lanzado tres de los cinco modos del inyector con `vsim -do "set ERR <modo>; do SimErr.do"`: uno que estropea un *flag*, uno que corrompe la máquina de estados y uno que cambia la salida en un ciclo sin operación. En los tres el scoreboard da el primer error en el flanco siguiente a la inyección y el informe final da **RESULTADO: FALLO** (tabla 7).

Tabla 7: Errores inyectados y respuesta del banco.

Modo	Error forzado	Resultado
FLAG	F_FULL_N = 0 a los 851 ns, con USE_DW = 1	“Error llenado” a los 950 ns. FALLO: 187 errores en F_FULL_N de 2550 comparaciones; también salta la aserción sobrellenado de la FIFO.
ESTADO	FSM forzada a LLEN0 con la FIFO a media ocupación	FALLO: 2774 errores en 7300 comparaciones (USE_DW 1218, DATA_OUT 1211, F_FULL_N 337, F_EMPTY_N 8).
DATOIDLE	DATA_OUT pasa de 0x78 a 0x79 a los 1651 ns, sin lectura ni escritura	“Salida diferente: obtenido=79 esperado=78” a los 1750 ns. FALLO.

En el modo **FLAG** el error dura mucho más que el ciclo forzado. En nuestra FIFO **F_FULL_N** se asigna en un **always_comb** que solo depende del estado, y tras un **release** una variable conserva el valor forzado hasta que su proceso vuelve a ejecutarse. Así que **F_FULL_N** sigue a 0 hasta que la FSM cambia de estado, 187 ciclos después (figura 14). Mientras tanto el generador sigue escribiendo, porque no mira el *flag*, y por eso salta también la aserción **sobrellenado** de la propia FIFO. El scoreboard cuenta un error en cada uno de esos ciclos, que es lo que esperábamos.

En el modo ESTADO la FIFO cree que está llena e ignora escrituras, así que se desincroniza del modelo y los errores se acumulan en todas las salidas. El test se alarga hasta 7300 ciclos porque la cobertura se mide con lo que observa el monitor, y una FIFO que no sigue a los estímulos tarda más en recorrer todos los bins. El modo DATOIDLE es el que demuestra que comparamos también los ciclos sin operación: antes de ese cambio el error habría pasado sin detectar.

```
# [0] Inyeccion de errores activada: FLAG
# Secuencia 1: Llenado
# [851000] INYECCION FLAG: F_FULL_N forzado a 0 con USE_DW=1
# ** Error: Error llenado: obtenido=0 esperado=1
# Time: 950 ns Scope: utilidades_pkg.FIFO_Scoreboard__1.evaluator_resultados File: ../src/FIFO_Scoreboard.sv Line: 99
> # ** Error: Error llenado: obtenido=0 esperado=1
> # Time: 1050 ns Scope: utilidades_pkg.FIFO_Scoreboard__1.evaluator_resultados File: ../src/FIFO_Scoreboard.sv Line: 99
> # ** Error: estas escribiendo sobre una fifo llena
> # Time: 1050 ns Started: 1050 ns Scope: FIFO_tb.duv.duv.sobrellenado File: ../src/FIFO_sintetizable.sv Line: 195 Expr: ~READ
> # ** Error: Error llenado: obtenido=0 esperado=1
> # Time: 1150 ns Scope: utilidades_pkg.FIFO_Scoreboard__1.evaluator_resultados File: ../src/FIFO_Scoreboard.sv Line: 99
> # ** Error: Error llenado: obtenido=0 esperado=1
> # Time: 1250 ns Scope: utilidades_pkg.FIFO_Scoreboard__1.evaluator_resultados File: ../src/FIFO_Scoreboard.sv Line: 99
> # ** Error: Error llenado: obtenido=0 esperado=1
> # FIN DEL TEST
> # grado_llenado : 100.00 %
> # estados_normales : 100.00 %
> # casos_especiales : 100.00 %
> # Cobertura total ($get_coverage): 100.00 %
> # ===== INFORME DEL SCOREBOARD =====
> # Transacciones comparadas: 2550
> # DATA_OUT aciertos = 2550 errores = 0
> # F_EMPTY_N aciertos = 2550 errores = 0
> # F_FULL_N aciertos = 2363 errores = 187
> # USE_DW aciertos = 2550 errores = 0
> # Esperados pendientes de comparar: 1
> # ** Error: RESULTADO: FALLO - 187 errores en 2550 comparaciones
> # Time: 255750 ns Scope: utilidades_pkg.FIFO_Scoreboard__1.report File: ../src/FIFO_Scoreboard.sv Line: 128
> # =====
```

Figura 13: Modo FLAG: mensaje de inyección, errores del scoreboard y de la aserción `sobrellenado`, e informe final con RESULTADO: FALLO.

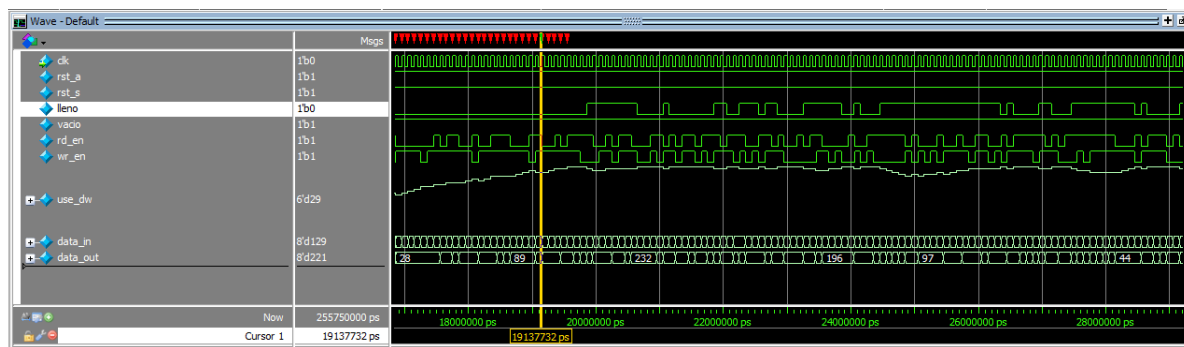


Figura 14: Modo FLAG en formas de onda: `lleno` (`F_FULL_N`) se queda a 0 hasta que la FSM cambia de estado, aunque `use_dw` está por debajo de 32. Las marcas rojas de arriba son los fallos de la aserción `sobrellenado`.

```
# ===== INFORME DEL SCOREBOARD =====
# Transacciones comparadas: 7300
# DATA_OUT aciertos = 6089 errores = 1211
# F_EMPTY_N aciertos = 7292 errores = 8
# F_FULL_N aciertos = 6963 errores = 337
# USE_DW aciertos = 6082 errores = 1218
# Esperados pendientes de comparar: 1
# ** Error: RESULTADO: FALLO - 2774 errores en 7300 comparaciones
# Time: 730750 ns Scope: utilidades_pkg.FIFO_Scoreboard__1.report File: ../src/FIFO_Scoreboard.sv Line: 128
# =====
```

Figura 15: Modo ESTADO: informe final con errores en las cuatro salidas.

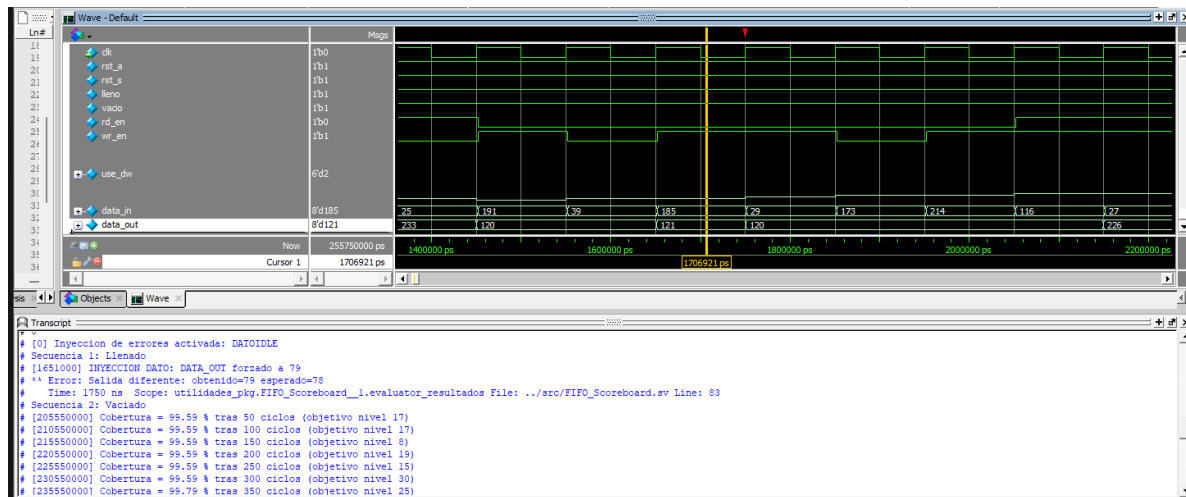


Figura 16: Modo DATOIDLE: `data_out` pasa de 120 a 121 (0x78 a 0x79) en un ciclo sin operación y el scoreboard lo detecta en el flanco siguiente.

5.5. FIFO del profesor

Con `vsim -do "set PROFESOR 1; do Sim.do"` el mismo banco verifica `FIFO_no_sintetizable`. El resultado es idéntico al de nuestra FIFO: 2550 comparaciones de las cuatro salidas sin errores y cobertura del 100% en el mismo número de ciclos (figura 17). Como la semilla es la misma, las dos FIFOs reciben exactamente los mismos estímulos, y que el scoreboard no encuentre ninguna diferencia en ninguna de las dos confirma que nuestra FIFO se comporta como el modelo de referencia.

```

# FIN DEL TEST
# grado_llenado : 100.00 %
# estados_normales : 100.00 %
# casos_especiales : 100.00 %
# Cobertura total ($get_coverage): 100.00 %
# ===== INFORME DEL SCOREBOARD =====
# Transacciones comparadas: 2550
# DATA_OUT aciertos = 2550 errores = 0
# F_EMPTY_N aciertos = 2550 errores = 0
# F_FULL_N aciertos = 2550 errores = 0
# USE_DW aciertos = 2550 errores = 0
# Esperados pendientes de comparar: 1
# RESULTADO: EXITO - 2550 comparaciones sin errores
# =====
# Simulación finalizada exitosamente.
# ** Note: $finish : ../src/FIFO_tb.sv(47)
# Time: 255750 ns Iteration: 3 Instance: /FIFO_tb

```

Figura 17: Informe final con la FIFO del profesor como DUV: 2550 comparaciones sin errores y cobertura del 100%.

5.6. Parametrización

Los parámetros se cambian al lanzar la simulación, sin tocar el código (`vsim -do "set WIDTH 16; set DEPTH 16; do Sim.do"`). `Regresion.do` lanza las tres combinaciones con las dos FIFOs y resume el resultado. Los seis casos dan EXITO (figura 20), y en cada configuración las dos FIFOs dan exactamente el mismo número de comparaciones sin errores, así que también con otros parámetros nuestra FIFO se comporta igual que la del profesor. La tabla 8 recoge el detalle con nuestra FIFO.

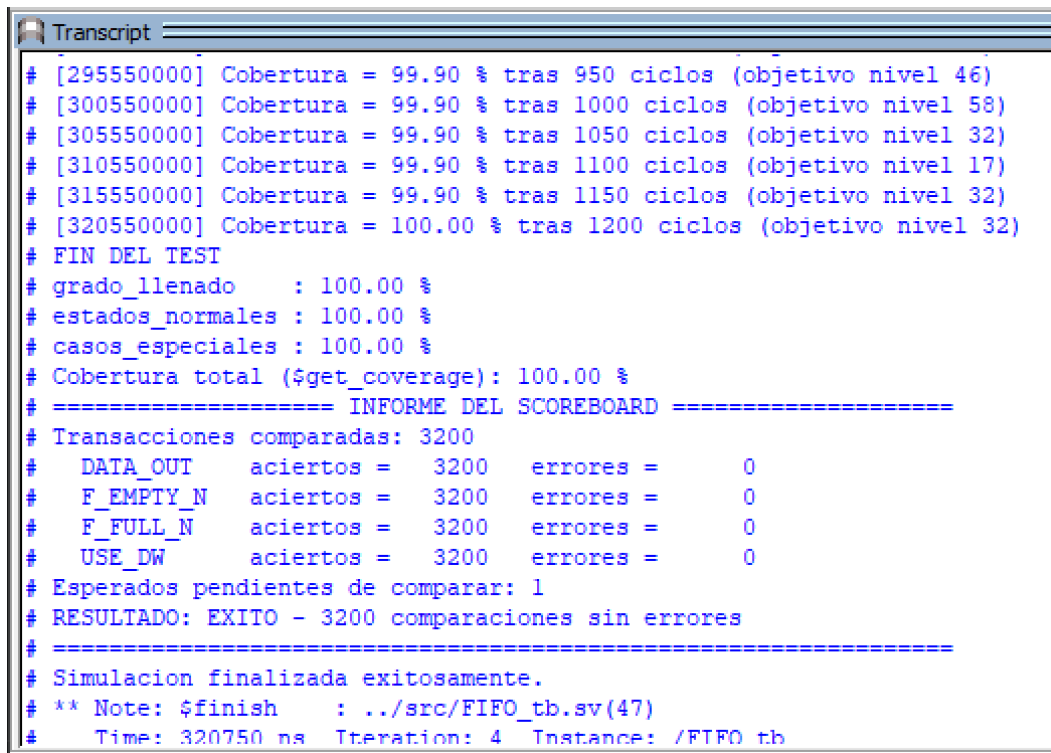
Tabla 8: Resultados con distintos parámetros (FIFO propia).

WIDTH	DEPTH	Comparaciones	Errores	Cobertura	Resultado
8	32	2 550	0	100 %	EXITO
16	16	2 000	0	100 %	EXITO
4	64	3 200	0	100 %	EXITO

El número de ciclos cambia con DEPTH, como era de esperar. Con 16 posiciones las dos fases fijas ya bastan para el 100 % y el bucle no llega a ejecutarse (2 000 ciclos). Con 64 hay 65 niveles que cubrir y el bucle necesita 24 ráfagas (1 200 ciclos más, 3 200 en total). Como los bins de `grado_llenado` se definen con DEPTH, el modelo de cobertura se adapta solo a cada configuración.

```
# Secuencia 2: Vaciado
# FIN DEL TEST
# grado_llenado      : 100.00 %
# estados_normales  : 100.00 %
# casos_especiales  : 100.00 %
# Cobertura total ($get_coverage): 100.00 %
# ===== INFORME DEL SCOREBOARD =====
# Transacciones comparadas: 2000
#   DATA_OUT   aciertos =   2000   errores =     0
#   F_EMPTY_N   aciertos =   2000   errores =     0
#   F_FULL_N    aciertos =   2000   errores =     0
#   USE_DW      aciertos =   2000   errores =     0
# Esperados pendientes de comparar: 1
# RESULTADO: EXITO - 2000 comparaciones sin errores
# =====
# Simulacion finalizada exitosamente.
# ** Note: $finish      : ../src/FIFO_tb.sv(47)
#   Time: 200750 ns   Iteration: 4   Instance: /FIFO_tb
```

Figura 18: Informe final con WIDTH = 16 y DEPTH = 16: 100 % de cobertura con las fases fijas y 2 000 comparaciones sin errores.



```

# [295550000] Cobertura = 99.90 % tras 950 ciclos (objetivo nivel 46)
# [300550000] Cobertura = 99.90 % tras 1000 ciclos (objetivo nivel 58)
# [305550000] Cobertura = 99.90 % tras 1050 ciclos (objetivo nivel 32)
# [310550000] Cobertura = 99.90 % tras 1100 ciclos (objetivo nivel 17)
# [315550000] Cobertura = 99.90 % tras 1150 ciclos (objetivo nivel 32)
# [320550000] Cobertura = 100.00 % tras 1200 ciclos (objetivo nivel 32)
# FIN DEL TEST
# grado_llenado      : 100.00 %
# estados_normales  : 100.00 %
# casos_especiales   : 100.00 %
# Cobertura total ($get_coverage): 100.00 %
# ===== INFORME DEL SCOREBOARD =====
# Transacciones comparadas: 3200
#   DATA_OUT   aciertos =   3200   errores =     0
#   F_EMPTY_N   aciertos =   3200   errores =     0
#   F_FULL_N    aciertos =   3200   errores =     0
#   USE_DW      aciertos =   3200   errores =     0
# Esperados pendientes de comparar: 1
# RESULTADO: EXITO - 3200 comparaciones sin errores
# =====
# Simulacion finalizada exitosamente.
# ** Note: $finish      : ../src/FIFO_tb.sv(47)
#   Time: 320750 ns   Iteration: 4   Instance: /FIFO tb

```

Figura 19: Informe final con WIDTH = 4 y DEPTH = 64: el bucle llega al 100 % tras 1 200 ciclos y hay 3 200 comparaciones sin errores.



```

# ===== RESUMEN DE LA REGRESION =====
#   FIFO propia      8 x 32   EXITO - 2550 comparaciones sin errores
#   FIFO propia     16 x 16   EXITO - 2000 comparaciones sin errores
#   FIFO propia      4 x 64   EXITO - 3200 comparaciones sin errores
#   FIFO profesor    8 x 32   EXITO - 2550 comparaciones sin errores
#   FIFO profesor   16 x 16   EXITO - 2000 comparaciones sin errores
#   FIFO profesor    4 x 64   EXITO - 3200 comparaciones sin errores
# =====

```

Figura 20: Resumen de Regresion.do: las dos FIFOs con las tres combinaciones de parámetros, todas con RESULTADO: EXITO.

6. Conclusiones

Nuestra FIFO sintetizable se comporta ciclo a ciclo igual que el modelo del profesor: con los mismos estímulos, las dos dan 2550 comparaciones de las cuatro salidas sin ningún error. La cobertura funcional llega al 100 % en 2550 ciclos gracias al test dirigido por cobertura, mientras que el test de duración fija se quedaba en el 99,79 % con 10 000 ciclos y el test dirigido del profesor en el 96,16 %.

El banco detecta los tres errores que hemos inyectado en el flanco siguiente a la inyección, incluido un cambio de DATA_OUT en un ciclo sin operación, lo que indica que de verdad observamos todas las salidas en todos los ciclos. Además, funciona sin cambios con las dos FIFOs y con 8×32, 16×16 y 4×64: la regresión completa da los seis casos sin errores, eligiendo la FIFO y los parámetros desde los scripts.

Veredicto: con estos resultados, como equipo de verificación damos el visto bueno a FIFO_sintetizable.